1. Измененная структура:

create table clients (

client_id number primary key, -- уникальный идентификатор рекомендуется сделать первичным ключом

client_name varchar(255), -- поле корректно

client_surname varchar(255), -- поле корректно

client_gender varchar(10), -- добавить пол для формирования предложения продуктов

login varchar(30) unique, --как правило, логин клиента в магазине должен быть уникальным

client_email varchar(255), -- добавить контактные данные клиента для настройки рекламы

client_phone_number varchar(255), -- добавить контактные данные клиента для настройки рекламы

city_id number, -- поле корректно, если существует таблица-справочник городов с id и названиями

birth_date date, -- добавить дату рождения

age number, -- поле само по себе некорректно, так как возраст-динамическая переменная. Можно оставить, если вычислять возраст как extract(year from age(now(), birth_date)) as age

reg_date date – поле коректно,

);

2. Я бы предложила создать таблицу-справочник с ценами PRICES со следующей структурой:

create table prices (

item_id number, -- id товара

price_id number primary key, -- id записи цены

item_cost number, -- стоимость товара

item_cost_dif number, -- процент изменения цены по сравнению с прошлым значением

cost_dif_type varchar(255), -- тип скидки

start_date date, -- дата начала действия цены

end_date date, -- дата завершения действия цены

foreign key (item_id) references items(item_id)

);

Актуальное значение цены из этого справочника (с самой поздней датой создания и пустой датой завершения) будет отражаться в ITEMS:

create table items (

item_id number primary key, -- уникальный id товара

item_name varchar(255), -- наименование товара

price_id number primary key, -- id записи цены

item_cost number -- текущая стоимость товара

);

3. Ошибки: отсутствует as при присвоении алиаса, нет явного указания таблиц, из которых выбираются атрибуты (особенно критично для department_id), неправильно указано название таблицы employees, соединение таблиц должно осуществляться при помощи оператора join on, условия задаются после оператора where, проверка на пустое значение задается как is null, при наличии агрегатной функции другие атрибуты в select

должны также войти в агрегатные функции или group by, для отбора условие для суммы, вычисляемой с помощью агрегата, нужно указать в операторе having, а не where.

```
SELECT D.DEPARTMENT_ID,
       D.DEPARTMENT_NAME,
       SUM(E.SALARY) AS TOTAL_SALARY
FROM EMPLOYEES E
JOIN DEPARTMENTS D
ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE E.MANAGER_ID IS NULL
  AND D.LOCATION_ID = (SELECT LOCATION_ID
 FROM LOCATIONS
WHERE CITY = 'SEOUL')
GROUP BY  D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING SUM (E.SALARY) >= 100000
```

4. Перепрыгнуть карандаш не получится, если его положили в угол или под стену, также если, например, он лежит на полу под столом, или прыгающий находится в другой части комнаты и не сможет допрыгнуть физически, или потолок в комнате настолько низкий, что можно только проползти над карандашом.